

interCEde

Design and Implementation of a Resource Access Layer for DiracX Internship plan



Alexander Auer
Student Number: s1152698

CERN LHCb Experiment DiracX
Hogeschool Leiden
Informatica

Internship Period: 01-09-2026 –
28-02-2027
Version: 1.0.0
Version Date: September 7, 2026



Contents

1	Introduction	2
2	The organisation	2
2.1	My supervisors	3
2.2	Stakeholders	3
3	Context	4
3.1	Competencies	4
4	Problem	5
5	Project goal	5
5.1	What	5
5.2	Why	5
5.3	How	6
6	Project boundaries and conditions	6
6.1	Scope	6
6.1.1	Case 1: HTCondor	6
6.1.2	Case 2: Multiple Resource Systems	7
6.1.3	Case 3: Composed Access and New Resource Types	7
6.2	Dependencies and Required Resources	7
6.2.1	Relevant Literature	7
6.2.2	DiracX Requirements	7
6.2.3	Batch systems	7
6.2.4	Grid Environment	8
6.3	Required Resources	8
7	Research	8
7.1	Main Research Question	8
7.2	Sub-Questions	8
7.3	How Will I Approach This Research	8
8	Deliverables / Data Points / PBS	9
8.1	InterCEde Product	9
8.2	Common Interface	9
8.3	Resource-Management Implementations	9
8.4	Testing and Evaluation	9
8.5	Data Points	10
8.6	Product Breakdown Structure	10
9	Project Approach	10
9.1	Activities	10
9.2	Personal Learning Goals	11
10	Project Planning	11
10.1	Presumptions	11
10.2	Vacation Time	11

11 Risks	11
11.1 Risk 1: Increasing Architectural Complexity	11
11.2 Risk 2: Differences Between batch systems	11
11.3 Risk 3: Insufficient Evaluation Results	12
12 Project handover	12
12.1 Transfer of deliverables	12
12.2 Archiving	12
12.3 Handover procedure	12
13 Attachments & figures	13
References	15
Glossary	16

1 Introduction

This internship plan describes the internship project carried out within the LHCb experiment at CERN as part of the EP-LCB-CS section. The project focuses on the design and implementation of interCEde, a resource access layer between distributed and heterogeneous computing resources (Worldwide lhc computing grid, WLCG) and DIRAC. This document describes the organisation and project context, the problem being addressed, the intended project goal and scope, the planned research, deliverables, approach, planning and risks. It also describes how the resulting work will be evaluated and handed over to the internship organisation.

2 The organisation

CERN is a large international research organisation consisting of many different departments and experiments. Because of the size of the organisation, this section will focus primarily on the parts of CERN that are relevant to my internship. The LHCb experiment is part of CERN's Experimental Physics (EP) department. LHCb uses the Large Hadron Collider (LHC) to investigate differences between matter and antimatter. One of the main goals of the experiment is to improve our understanding of why the universe contains significantly more matter than antimatter, despite matter and antimatter being expected to have been produced in approximately equal amounts during the early universe. (CERN., n.d.)

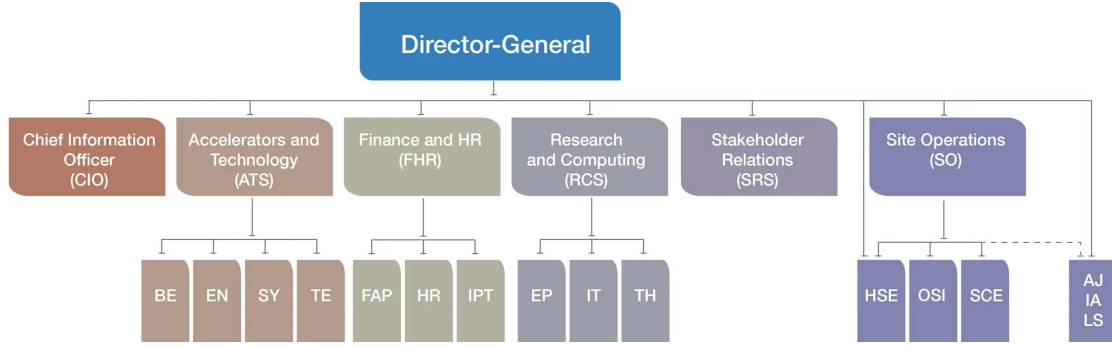


Figure 1: Management structure of CERN

My work is carried out within the software development activities related to the LHCb experiment. The department and teams involved are responsible for developing and maintaining software used to process and manage the data produced by the experiment. Within this environment, I am working as part of the DIRAC team. My internship project focuses on the development of interCEde, a resource access layer intended to provide an interface between the WLCG and DIRAC.

2.1 My supervisors

My supervisor is Alexandre Boyer, who is part of the DIRAC team and is responsible for coordinating the team’s sprint meetings. At the beginning of my internship, we agreed that Alexandre would check in with me on a daily basis. These check-ins provide an opportunity to discuss my progress, ask questions and address any issues that arise during the development of the project.

In addition to the daily check-ins, I participate in the team’s weekly meeting on Thursday mornings. During this meeting, I can discuss my progress with the rest of the team and stay informed about developments within the project and the wider team.

My university supervisor is Wytze Breukel. I will communicate with him about the scope and direction of my internship project and research. He will provide guidance from the perspective of the Informatica programme and help ensure that the project remains aligned with the requirements and intended learning outcomes of the internship.

2.2 Stakeholders

The primary stakeholders are the developers and maintainers of DIRAC, DiracX, and community-specific extensions such as LHCbDIRAC. The project is primarily intended to benefit the LHCbDIRAC team, which maintains, operates, and develops the LHCb-specific extension of DIRAC, while also contributing to the wider DIRAC ecosystem through reusable components and interfaces. In particular, the project is expected to facilitate the migration from DIRAC to DiracX and provide a maintainable and reusable interface for accessing different batch systems.

My company supervisor, Alexandre Boyer, is also a stakeholder, as he provides guidance on the technical direction and scope of the project and represents the interests of the development team within the project.

Another group of stakeholders consists of developers and maintainers across the DIRAC consortium. The design of the resource access layer may affect how DIRAC-based and DiracX services interact with underlying batch systems. Providing a common interface for different resource systems should make it easier for developers to modify and debug

these interactions, while also allowing changes to be tested more reliably before they affect users.

The wider group of stakeholders consists of users of DIRAC-based services, including LHCb physicists. These users generally do not interact directly with the underlying resource-access interfaces, but they may nevertheless be affected by changes or bugs in the infrastructure. By enabling developers to detect and diagnose problems earlier, the project should indirectly improve the reliability of the services experienced by end users.

The main stakeholders are visualised in Figure 3.

3 Context

From discussions with my supervisor, I learned that the DIRAC developers are currently working on migrating the existing DIRAC services to DiracX. DIRAC has a distributed, service-oriented architecture in which individual services run as separate processes across different virtual machines. Although this shares characteristics with a microservice architecture, many DIRAC services are stateful and therefore cannot be easily replicated or scaled horizontally.

The migration to DiracX addresses several limitations that have emerged from many years of development and maintenance of the DIRAC codebase. Its resource access interfaces contain numerous special cases for different types of Computing Elements (CEs), making them difficult to abstract, maintain, and extend. Furthermore, there is currently no practical way to systematically test changes to these interfaces against the different computing resources they support.

DiracX instead uses a monolithic, stateless application deployed through Kubernetes. Each pod can provide the complete set of services, allowing the application to be replicated and scaled horizontally. To support this architecture, the interaction with external computing resources is separated into a dedicated resource access layer.

This project develops *interCEde*, a library providing a maintainable and testable interface between DiracX and the heterogeneous batch systems used by the DIRAC consortium. The project focuses on analysing the existing resource access implementations and the requirements of the supported batch systems, investigating suitable architectural approaches, and using these findings to design and implement the resource access layer. The resulting implementation is evaluated against the identified requirements and criteria for maintainability and testability.

3.1 Competencies

The internship provides opportunities to demonstrate competencies in **analysis** and **realisation**.

Analysis is demonstrated through examining the existing DIRAC resource access layer, the requirements of the migration to DiracX, and the characteristics of the supported batch systems. This analysis is used to establish requirements and design criteria for *interCEde* and to investigate suitable software architecture and design approaches.

Realisation is demonstrated through implementing the resulting design as working software. This includes developing the common resource access interface and its implementations, initially starting with HTCondor, while allowing for additional batch systems. Automated tests are developed alongside the implementation to verify its behaviour and support its maintainability. The resulting software is evaluated against the requirements and design criteria established during the analysis.

4 Problem

The current implementation of resource access in DIRAC reflects the heterogeneous nature of the WLCG. Different batch systems are supported through separate classes and implementations within the DIRAC codebase. While this provides a unified interface from a user's perspective, it also results in a very hard to maintain codebase for the developers. Consequently, changes to one part of the system can have unintended effects on other components, making the codebase difficult to maintain and extend. (DIRACGrid, n.d.-a)

Based on an initial investigation of the existing implementation and discussions with the Dirac/DiracX team, it became clear that the differences between batch systems need to be hidden from the services that use them. At the same time, the interface needs to remain flexible enough to support different types of resources without requiring major changes to the rest of the system. These observations form the basis for investigating alternative designs for the resource access layer.

With the migration towards DiracX, there is an opportunity to reconsider how access to the different batch systems is structured. The main problem addressed by this project is therefore how *interCEde* can be designed to provide a suitable abstraction between the grid and DiracX while remaining maintainable and adaptable to different batch systems.

To investigate this problem, different design approaches will be explored and compared. These approaches will be evaluated against criteria such as maintainability, testability, extensibility, and suitability for integration with DiracX. Based on this comparison, an appropriate approach will be selected and implemented as working software. The resulting implementation will then be evaluated to determine whether the chosen design provides a suitable foundation for supporting the different batch systems.

5 Project goal

5.1 What

The goal of this project is to design and implement *interCEde*, a resource access layer that provides an interface between DiracX and heterogeneous computing resources. Its purpose is to abstract the differences between the various batch systems, cloud providers, and computing elements used to execute workloads, allowing DiracX services to interact with these resources through a common interface.

The resources accessed through *interCEde* are not limited to those participating in the WLCG. In addition to grid resources, LHCb makes use of opportunistic resources that are operated outside the WLCG. While sites participating in the WLCG are subject to requirements and policies associated with their level of participation, opportunistic resources are independently operated and may impose their own access and usage requirements. *interCEde* should therefore provide a uniform interface regardless of whether a resource is part of the WLCG or is provided opportunistically.

5.2 Why

The development of *interCEde* contributes to the broader migration from the existing DIRAC system to DiracX. The current implementation of resource access is closely coupled to the different batch systems supported by the WLCG. This makes the system more difficult to maintain and extend when changes or additional batch systems are introduced.

By separating resource access from the services that use it, *interCEde* aims to provide a more maintainable and adaptable foundation for interacting with the heterogeneous

resources of the grid. This is particularly relevant as DIRAC services are migrated to the newer DiracX architecture.

5.3 How

The project will begin by investigating different approaches for designing the resource access layer. These approaches will be compared using criteria such as maintainability, testability, extensibility, and suitability for integration with DiracX.

Based on the results of this investigation, an appropriate design will be selected and implemented as working software. The implementation will initially focus on providing a common interface and supporting HTCondor (HTCondor Project, 2026). If sufficient progress is made, support for additional batch systems will be added. The resulting implementation will then be tested and evaluated against the criteria established during the research.

6 Project boundaries and conditions

6.1 Scope

The internship is officially scheduled for a period of five months. However, I intend to continue working on the project for an additional month after the official internship period. To ensure that a demonstrable result is achieved within the five-month period, the scope of the project will be adjusted according to the available time and progress. The expected work after the internship will also be described in this chapter.

The complete development of interCEde is considered too ambitious to be fully realised within the available time. Therefore, in consultation with Alexandre, the project will initially focus on designing the common interface and implementing it for HTCondor. From this starting point, three possible cases have been defined. Each case represents an increasing level of functionality, while also providing an opportunity to validate the design choices made during the research.

The research conducted during this project will focus on identifying design choices that contribute to maintainable and testable software. These findings will be used to guide the design of interCEde rather than treating the research and implementation as separate activities. The resulting design will be implemented and evaluated throughout the project. The cases therefore represent not only increasing implementation scope, but also increasing opportunities to assess whether the chosen design remains suitable as the complexity of the resource access layer increases. Case 1 constitutes the minimum scope, while Cases 2 and 3 are pursued only if project progress permits.

6.1.1 Case 1: HTCondor

This case represents the minimum intended result of the project, and will consist of designing the common interface and implementing it for HTCondor. The design will be based on the findings of the research into maintainability and testability. The resulting implementation will provide a functional resource access layer capable of interacting with HTCondor through the newly designed interface.

It will establish the foundation of interCEde and provide an initial implementation against which the design choices can be evaluated. In particular, the maintainability and testability of the resulting code will be assessed through the chosen evaluation criteria.

6.1.2 Case 2: Multiple Resource Systems

The second case extends Case 1 by adding support for Slurm and ARC. Supporting multiple resource-management systems will allow the design to be evaluated against a more heterogeneous set of requirements. (SchedMD, n.d.), (Nordugrid Collaboration, n.d.)

The implementation of these additional systems will be used to determine whether the design choices identified during the research remain effective when differences between resource systems need to be accommodated. This case therefore provides a stronger validation of the maintainability and testability of the common interface.

6.1.3 Case 3: Composed Access and New Resource Types

The third case extends Case 2 by adding support for composed access paths and a new type of resource-management functionality. A composed access path occurs when multiple access mechanisms need to be combined to reach a resource. For example, a job may require an SSH connection before it can access a Slurm resource. (DIRACGrid, 2026)

In addition, this case will introduce support for Kubernetes. Unlike HTCondor, Slurm, and ARC, Kubernetes represents a new type of resource-management functionality within interCEde. Its implementation will therefore provide an opportunity to evaluate whether the design is sufficiently extensible to accommodate functionality that differs substantially from the initially supported systems. (Kubernetes Authors, n.d.)

Case 3 represents the most extensive scope of the project. It will only be pursued if sufficient progress has been made on the preceding cases and if the research and implementation provide a suitable basis for extending the system. The additional month following the official internship period can be used to work towards this case if necessary.

6.2 Dependencies and Required Resources

The development of interCEde depends on several external factors that may influence the implementation and achievable scope of the project.

6.2.1 Relevant Literature

The project depends on the availability of sufficient literature on software maintainability, testability, and software architecture. This literature will provide the theoretical foundation for the design choices and evaluation criteria used throughout the project.

6.2.2 DiracX Requirements

The common interface depends on the requirements and job structure defined by DiracX, based on discussions with the DiracX team. These requirements determine which functionality the resource access layer must provide and may therefore influence the design of the interface.

6.2.3 Batch systems

The implementation depends on the capabilities and interfaces of the batch systems supported by the project. In particular, the available functionality and documentation for HTCondor, Slurm, ARC, and Kubernetes will influence the implementation and the evaluation of the proposed design.

6.2.4 Grid Environment

The project also depends on the software and infrastructure available on the target grid. In particular, the supported versions and capabilities of the batch systems, as well as access to suitable computing resources for testing, may constrain the implementation.

Additional dependencies may become apparent during the project as the interface and its implementations are developed. These will be documented and considered when determining which scope case can be achieved within the available time.

6.3 Required Resources

The project requires access to the relevant DiracX and grid environments, including suitable test resources and supported versions of the required batch systems. A development environment with the necessary software, libraries, and testing tools will also be required. Access to relevant literature, documentation, and representative jobs will be needed to guide the design and evaluate the resulting implementation.

7 Research

To support the design of a maintainable and testable resource access layer, research will be conducted into software architecture, maintainability, testability, and the requirements of different batch systems. The research will be guided by a main research question and several sub-questions. The findings will be used to establish design criteria for interCEde and to evaluate the resulting implementation.

7.1 Main Research Question

How can a resource access layer for interCEde be designed to provide a maintainable and testable common interface for different batch systems, cloud providers and computing elements?

7.2 Sub-Questions

- What software design principles and architectural patterns contribute to maintainable and testable software?
- To what extent does the current DIRAC resource access layer satisfy these principles, and where does it fall short?
- What differences between the batch systems need to be considered when designing a common resource access interface?
- How can the differences between batch systems be accommodated in a common interface while satisfying the identified requirements for maintainability and testability?
- To what extent does the resulting design and implementation satisfy the identified requirements and design criteria?

7.3 How Will I Approach This Research

- **Literature review:** Reviewing academic and professional literature on software architecture, maintainability, and testability to identify relevant principles and architectural patterns. Findings from multiple sources will be compared and translated into concrete design criteria.

- **System analysis:** Analysing the requirements and characteristics of the batch systems supported by interCEde to identify differences that could influence the common interface.
- **Design and implementation:** Applying the identified design criteria to the interCEde architecture and validating them through implementation, starting with HTCondor and extending to additional systems where the project scope allows.
- **Evaluation:** Evaluating the resulting implementation against the established design criteria using automated tests and the practical experience gained from supporting different batch systems. The findings will be used to refine the design where necessary.

8 Deliverables / Data Points / PBS

8.1 InterCEde Product

The main objective of the project is to design and implement interCEde, a common resource access layer for DiracX. The complete product will provide a consistent interface for interacting with different batch systems while focusing on maintainability and testability.

The complete implementation is divided into several deliverable groups.

8.2 Common Interface

The common interface forms the foundation of interCEde and will define how DiracX interacts with different batch systems, cloud providers and computing elements.

- A technical design of the common interface and its architecture.
- A functional implementation of the interface.
- Research supporting the chosen design decisions.

8.3 Resource-Management Implementations

The interface will be implemented for different batch systems according to the project scope.

- A functional HTCondor implementation as the minimum intended result.
- Slurm and ARC implementations if the project progresses to Case 2.
- Kubernetes and composed access support if sufficient progress allows Case 3 to be pursued.

8.4 Testing and Evaluation

The resulting implementation will be evaluated to determine whether the design achieves the intended goals of maintainability and testability.

- An automated testing strategy using GitHub CI and Docker-based resource-management environments.
- Automated tests covering the common interface and its implementations.
- An evaluation of the implementation against the defined maintainability and testability criteria.
- An evaluation report documenting the results and conclusions.

8.5 Data Points

Several supporting deliverables will be produced throughout the project to document the development process and research.

- Research portfolio containing the relevant literature, analysis, and research findings.
- Technical and functional design documentation.
- Testing strategy and test results.
- Documentation of design decisions and their evaluation.
- Presentations and progress updates for the project team and supervisor.
- Feedback received during the design and implementation phases and how it was incorporated.

8.6 Product Breakdown Structure

The Product Breakdown Structure (PBS) provides an overview of the products developed during the project. The complete interCEde product is divided into the common interface, resource-management implementations, testing and evaluation, and supporting documentation. The structure is represented in Figure 4.

9 Project Approach

9.1 Activities

To achieve the deliverables outlined in Chapter 8, the project will be divided into four main activity clusters:

1. Research & Analysis (Weeks 1–4)

- Finalize the requirements and project scope.
- Research software maintainability, testability, and suitable architectural approaches.
- Analyse the existing DIRAC interface layer and the requirements of DiracX.
- Analyse the interfaces and capabilities of the supported batch systems.

2. Design (Weeks 5–8)

- Design the common interCEde interface based on the research findings.
- Define the architecture and structure of the resource access layer.
- Design the approach for supporting different batch systems.
- Review and refine the design based on the identified requirements.

3. Implementation & Testing (Weeks 9–20)

- Implement the common interface and HTCondor integration.
- Develop automated tests using Docker-based resource-management environments.
- Extend the implementation to Slurm and ARC if progress allows.
- Evaluate the maintainability and testability of the resulting implementation.

4. Evaluation, Documentation & Handover (Weeks 21–24)

- Evaluate the final implementation against the defined criteria.
- Finalize the implementation and technical documentation.
- Document the research findings, design decisions, and evaluation results.
- Prepare the project for handover and future development.

9.2 Personal Learning Goals

- Apply theoretical knowledge of software engineering and architecture to a real-world scientific computing project.
- Gain practical experience with batch systems and distributed computing environments.
- Develop experience in designing maintainable and testable software abstractions.
- Gain experience in evaluating and communicating architectural decisions.

10 Project Planning

10.1 Presumptions

The planning is based on the activity clusters described in Chapter 9 and the three implementation cases defined in the project scope. The initial focus will be on completing the common interface and HTCondor implementation. If sufficient progress is made, development will continue with Slurm and ARC, followed by Kubernetes and composed access paths if the scope allows.

The current planning is represented in the Gantt chart in Figure 2. The planning will be updated throughout the project to reflect the actual progress and any changes to the project scope.

10.2 Vacation Time

The project planning takes the planned vacation periods into account. The remaining weeks are considered available working weeks.

11 Risks

11.1 Risk 1: Increasing Architectural Complexity

Risk: The common interface becomes too complex as more batch systems are supported.

Impact: High – this could reduce the maintainability and testability of interCEde and result in a solution that does not improve upon the existing DIRAC interface layer.

Mitigation: Continuously compare the design with the research findings and evaluate it as each new resource-management system is added. Consult the supervisor when significant design issues arise.

11.2 Risk 2: Differences Between batch systems

Risk: The differences between batch systems may make it difficult to provide a suitable common abstraction.

Impact: High – the interface may require system-specific exceptions or become too restrictive to support all systems effectively.

Mitigation: Validate the interface incrementally by implementing HTCondor first, followed by Slurm and ARC. Use the differences between these systems to refine the abstraction before considering Kubernetes.

11.3 Risk 3: Insufficient Evaluation Results

Risk: The implementation may not provide enough evidence to determine whether the design improves maintainability and testability.

Impact: Medium – this could make it difficult to justify the chosen design and assess whether interCEde achieves its intended goals.

Mitigation: Define evaluation criteria early and apply them throughout development. Use automated testing through GitHub CI and Docker-based environments to consistently evaluate the implementations.

12 Project handover

12.1 Transfer of deliverables

I will deliver to CERN:

- A working version of the InterCEde library, including working testing environment in the GitHub CI.
- Design documents for InterCEde along with explanations of my design choices, with references to my research.
- My complete research documentation and the sources used.

12.2 Archiving

All my documents, code and research will be handed over to my internal supervisor.

12.3 Handover procedure

1. Hold final presentation with DIRAC team to explain my finished work.
2. Review documentation and code with supervisor.
3. Hand over all documentation and hand over remaining merge requests.
4. Confirm all materials are handed over with proper access.

13 Attachments & figures

Implementing interCEde

CERN

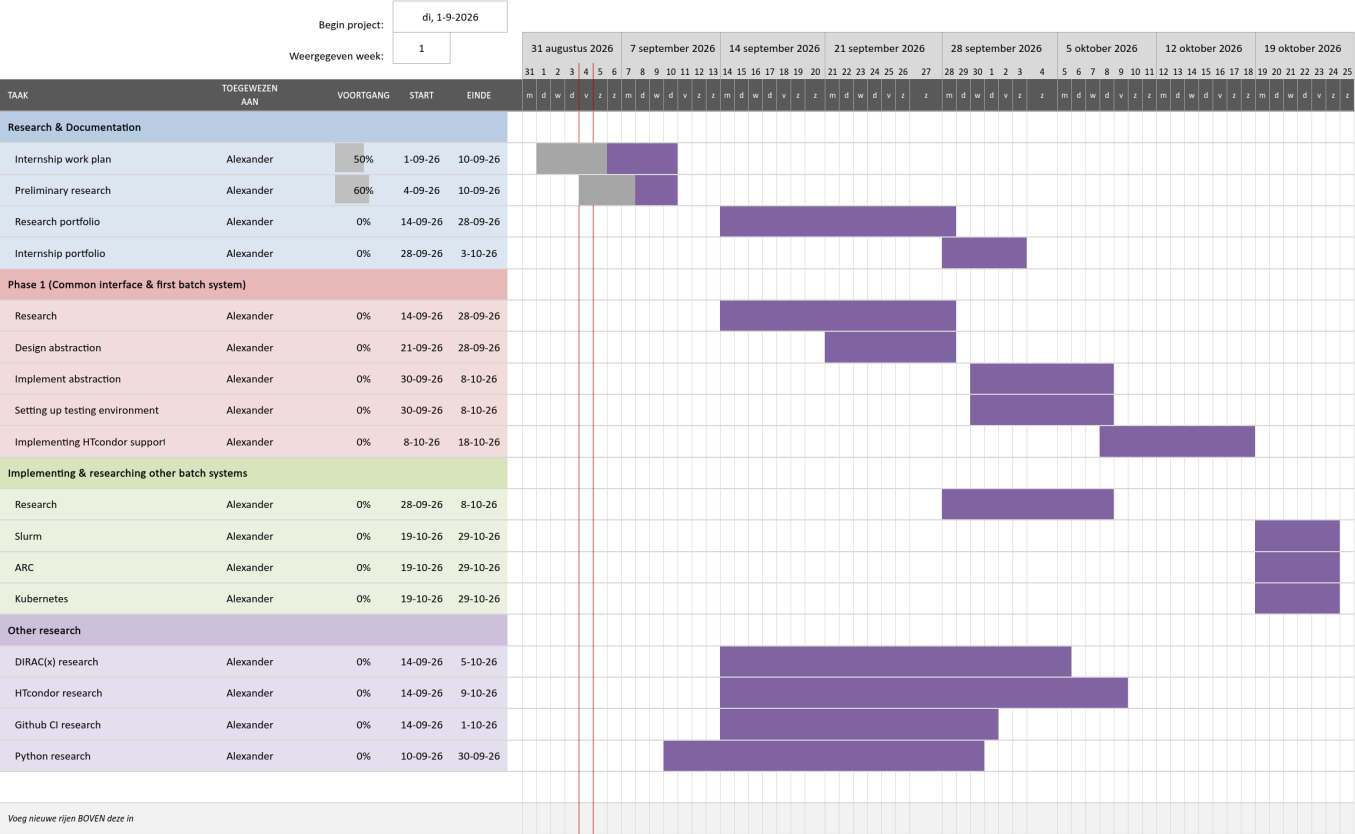


Figure 2: Gantt chart

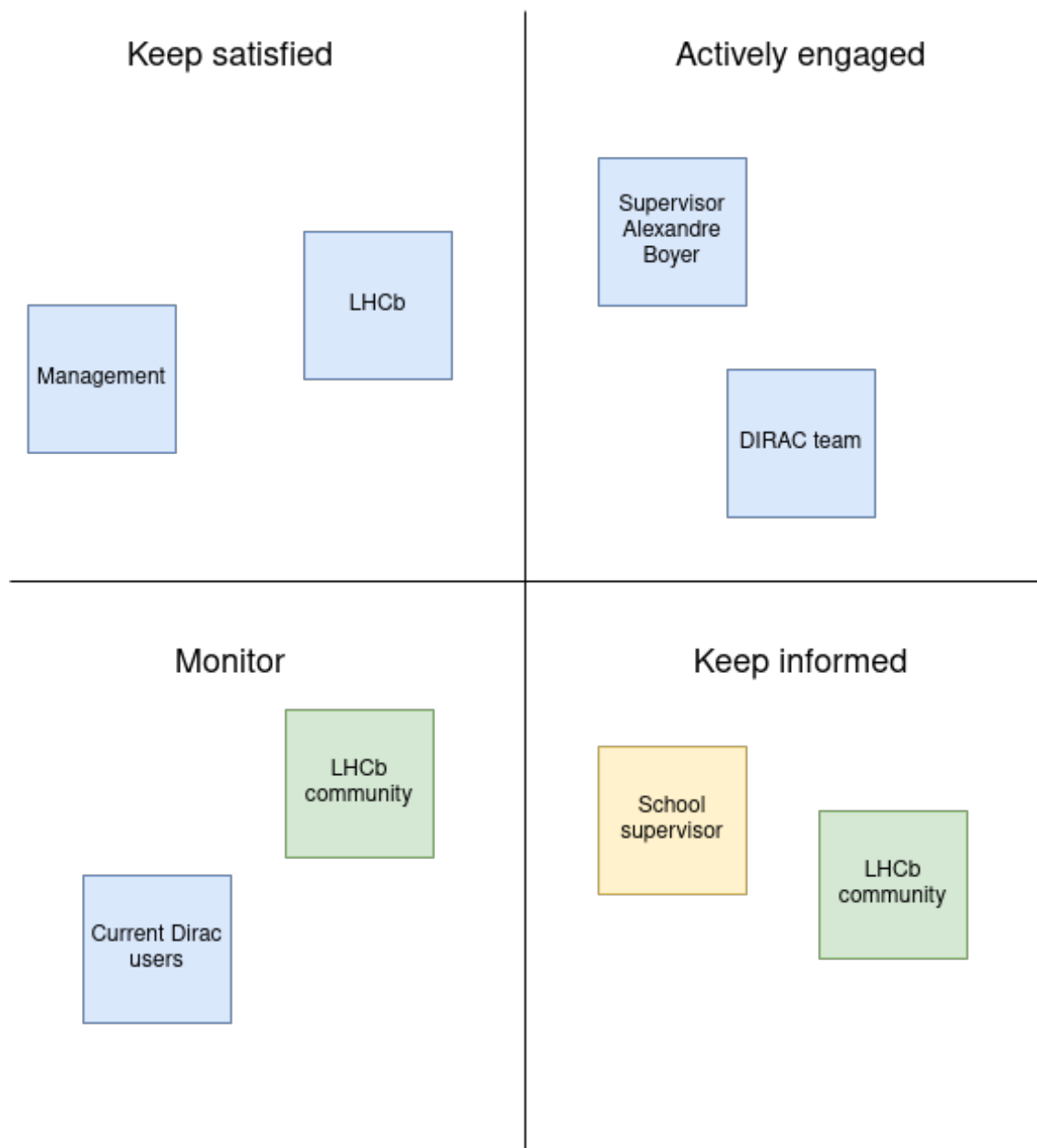


Figure 3: Stakeholder analysis

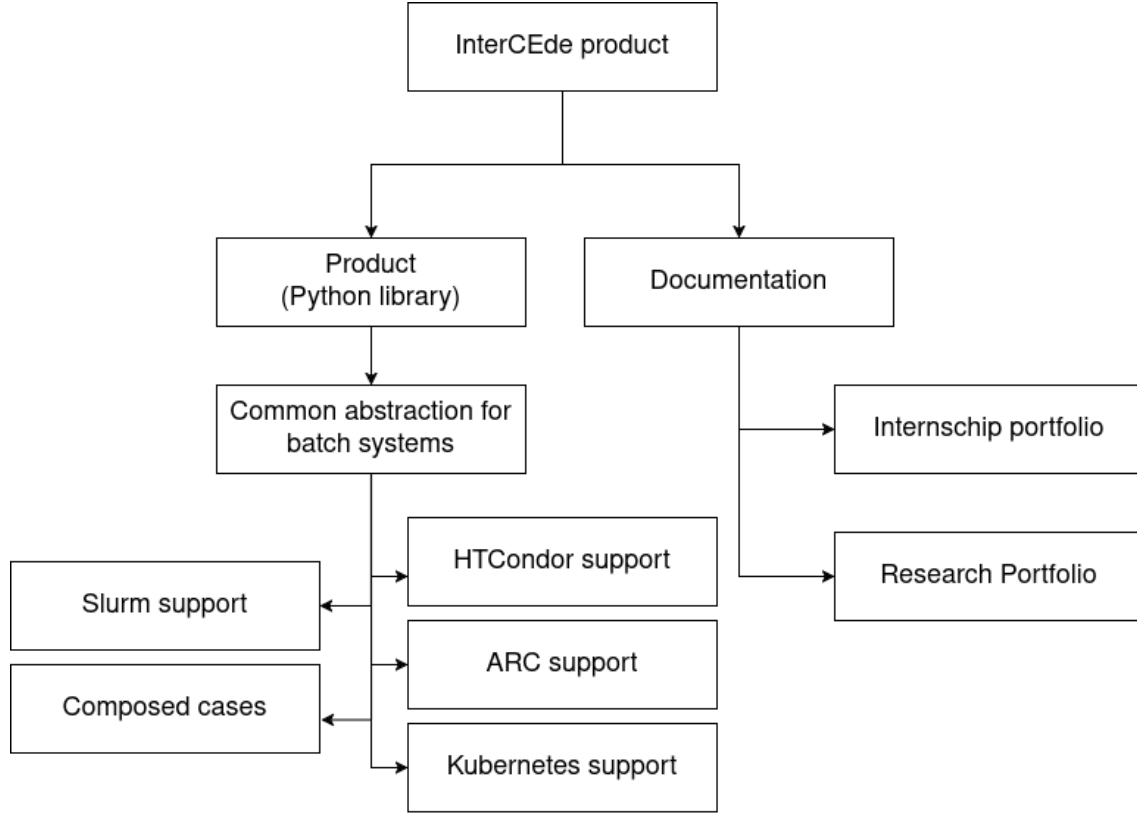


Figure 4: Product breakdown structure

References

- CERN. (n.d.). *Lhcb – home*. <https://home.cern/science/experiments/lhcb/>
- DIRAC Project. (n.d.). *Dirac*. <https://github.com/DIRACGrid>
- DIRACGrid. (2026). *SSHComputingElement.py*. Retrieved September 4, 2026, from <https://github.com/DIRACGrid/DIRAC/blob/integration/src/DIRAC/Resources/Computing/SSHComputingElement.py>
- DIRACGrid. (n.d.-a). *DIRAC/src/DIRAC/Resources/Computing at integration*. GitHub. <https://github.com/DIRACGrid/DIRAC/tree/integration/src/DIRAC/Resources/Computing>
- HTCondor Project. (2026). *Htcondor version 25.14.0 manual*. Retrieved September 4, 2026, from <https://htcondor.readthedocs.io/en/main/>
- Kubernetes Authors. (n.d.). *Kubernetes documentation*. Retrieved September 4, 2026, from <https://kubernetes.io/docs/home/>
- NorduGrid Collaboration. (n.d.). *Nordugrid arc 7 documentation*. Retrieved September 4, 2026, from <https://www.nordugrid.org/arc/arc7/>
- SchedMD. (n.d.). *Slurm workload manager documentation*. Retrieved September 4, 2026, from <https://slurm.schedmd.com/>

Glossary

DIRAC DIRAC provides a complete solution to one or more user community requiring access to distributed resources. DIRAC builds a layer between the users and the resources offering a common interface to a number of heterogeneous providers, integrating them in a seamless manner, providing interoperability, at the same time as an optimized, transparent and reliable usage of the resources (DIRAC Project, n.d.). 2

heterogeneous The characteristic of a computing network or grid consisting of interconnected nodes that feature diverse hardware architectures, operating systems, network protocols, and software configurations. 2, 5